Human Verification: `Containers.DContCat`

Directed Containers as Categories, and Cofunctors

Machine-generated from `lean/DContCat.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.DContCat` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **2**

2 Small categories **2**
2.1 Structure: `SmallCat` . . . 2

3 The dictionary: directed container ↔ small category **3**
3.1 Lemma: `transport_cancel` . . . 3
3.2 Lemma: `DirectedContainer.sub_shift_eq` . . . 3
3.3 Definition: `DirectedContainer.toSmallCat` . . . 3
3.4 Definition: `SmallCat.toDirectedContainer` . . . 4

4 The round-trip, honestly **5**
4.1 Definitions: `toRoundtripPos` and `ofRoundtripPos` . . . 5
4.2 Theorems: round-trip bijection . . . 5

5 Morphisms: directed-container morphisms and cofunctors **6**
5.1 Structure: `DContMorphism` . . . 6
5.2 Structure: `Cofunctor` . . . 6
5.3 Definitions and theorems: the hom-set bijection . . . 7

6 The category of directed containers (via cofunctors) **8**
6.1 Definition: `Cofunctor.id` . . . 8
6.2 Lemma: `Cofunctor.comp_transport` . . . 8
6.3 Definition: `Cofunctor.comp` . . . 8
6.4 Theorems: category laws for `Cofunctor` . . . 9

7 Directed-container morphisms as a category **9**
7.1 Definitions and theorems: `DContMorphism` as a category . . . 9

8 Translation conventions **10**

1 Overview

This file formalises the *object-level dictionary* of Ahman–Uustalu (2016): a directed container D presents a small category $\mathcal{C}(D)$, and conversely every small category presents a directed container. It also formalises the *morphism-level* result: directed-container morphisms are exactly **cofunctors** between the induced categories.

The key points:

- Objects of $\mathcal{C}(D)$ are shapes; morphisms $s \to s'$ are positions $p : P(s)$ with $s \downarrow p = s'$.
- The identity at s is the root $o(s)$; composition is the shift $\oplus$.
- Category laws are exactly D2/D3 (unit laws) and D5 (associativity).
- The bijection between directed-container morphisms and cofunctors is witnessed by `rfl` in both directions — the two structures carry literally the same data under different field names.

2 Small categories

2.1 Structure: `SmallCat`

Definition 2.1 (Small category)**.** A **small category** $\mathcal{C}$ consists of:

- a type Obj of *objects*;
- for each pair a, b : Obj, a type $\mathrm{Hom}(a, b)$ of *morphisms*;
- for each a, an *identity* $\mathrm{id}_a : \mathrm{Hom}(a, a)$;
- *diagrammatic composition* $\mathrm{comp} : \mathrm{Hom}(a, b) \to \mathrm{Hom}(b, c) \to \mathrm{Hom}(a, c)$ (left-to-right order, matching $\oplus$ in directed containers);

satisfying:

- **left unit:** $\mathrm{comp}(\mathrm{id}_a, f) = f$;
- **right unit:** $\mathrm{comp}(f, \mathrm{id}_b) = f$;
- **associativity:** $\mathrm{comp}(\mathrm{comp}(f, g), h) = \mathrm{comp}(f, \mathrm{comp}(g, h))$.

Lean.

```
structure SmallCat where
  Obj : Type
  Hom : Obj → Obj → Type
  id : (a : Obj) → Hom a a
  comp : {a b c : Obj} → Hom a b → Hom b c → Hom a c
  id_comp : ∀ {a b : Obj} (f : Hom a b), comp (id a) f = f
  comp_id : ∀ {a b : Obj} (f : Hom a b), comp f (id b) = f
  assoc : ∀ {a b c d : Obj} (f : Hom a b) (g : Hom b c) (h : Hom c d),
    comp (comp f g) h = comp f (comp g h)
```

Haskell.

```
-- A small category, hand-rolled.
-- Composition is in diagrammatic (left-to-right) order.
data SmallCat obj = SmallCat
  { hom     :: obj → obj → Type  -- not expressible in value-level
    Haskell
  , catId   :: obj → ()          -- identity (approximated)
  , catComp :: ()                -- composition (approximated)
```

```
  }
-- The laws are stated as properties (QuickCheck style):
-- prop_id_comp  f = comp (catId a) f == f
-- prop_comp_id  f = comp f (catId b) == f
-- prop_assoc f g h = comp (comp f g) h == comp f (comp g h)
```

Fidelity note. Lean's structure bundles the proof fields directly. Haskell cannot express dependent hom-types or carry proof fields in a data type without significant ceremony; the Haskell rendering approximates the data and states the laws as external properties.

3 The dictionary: directed container ↔ small category

3.1 Lemma: `transport_cancel`

Proposition 3.1 (Transport round-trip)**.** For any type family $P : \alpha \to \mathbf{Type}$ and equality $h : x = y$, transporting a term $p : P(y)$ back along h and then forward along h is the identity:

$$h \triangleright h^{-1} \triangleright p \;=\; p.$$

Lean.

```
theorem transport_cancel {alpha : Type} {P : alpha → Type} {x y :
    alpha}
    (h : x = y) (p : P y) : h [> h.symm [> p = p
```

Haskell.

```
-- Transport is trivial in Haskell (no dependent types).
-- The round-trip is the identity by definition.
```

3.2 Lemma: `DirectedContainer.sub_shift_eq`

Proposition 3.2 (Codomain of a composite morphism)**.** Let D be a directed container. If $p : P(a)$ with $a \downarrow p = b$, and $q : P(b)$ with $b \downarrow q = c$, then

$$a \downarrow (p \oplus q') \;=\; c,$$

where $q' = h_1^{-1} \triangleright q$ is q transported to live over $a \downarrow p$.

Lean.

```
theorem sub_shift_eq {a b c : D.Shape} (p : D.Pos a) (hp : D.sub a p =
    b)
    (q : D.Pos b) (hq : D.sub b q = c) :
    D.sub a (D.shift a p (hp.symm [> q)) = c
```

Haskell.

```
-- Follows immediately from D4 when hp and hq are rfl.
-- sub_shift_eq :: sub a (shift a p q) == c
-- given: sub a p = b, sub b q = c
```

3.3 Definition: `DirectedContainer.toSmallCat`

Definition 3.3 (Small category presented by a directed container)**.** Given a directed container $D = (S, P, o, \downarrow, \oplus)$, define a small category $\mathcal{C}(D)$ as follows:

$$\begin{aligned} \mathrm{Obj} &= S, \\ \mathrm{Hom}(a, b) &= \{p : P(a) \mid a \downarrow p = b\}, \\ \mathrm{id}_a &= (o(a),\ D_1(a)), \\ \mathrm{comp}(p, q) &= (p \oplus q',\ \texttt{sub_shift_eq}), \end{aligned}$$

where q' is q transported from the fibre over b to the fibre over $a \downarrow p$ via the equality p.property. The unit and associativity laws are D2/D3 and D5 respectively (after substituting the codomain equalities).

Lean.

```
def toSmallCat : SmallCat where
  Obj := D.Shape
  Hom a b := { p : D.Pos a // D.sub a p = b }
  id a := ⟨D.root a, D.d1 a⟩
  comp f g :=
    ⟨D.shift _ f.val (f.property.symm ▸ g.val),
     D.sub_shift_eq f.val f.property g.val g.property⟩
  id_comp := ...
  comp_id := ...
  assoc := ...
```

Haskell.

```
-- The category presented by a directed container.
-- Objects: shapes s
-- Morphisms s→t: positions p over s with sub s p = t
-- Identity at s: root s (with D1 saying sub s (root s) = s)
-- Composition (p, q) where sub s p = t: shift s p q
--   (q transported to live over sub s p)
```

3.4 Definition: `SmallCat.toDirectedContainer`

Definition 3.4 (Directed container presented by a small category)**.** Given a small category $\mathcal{C}$, define a directed container as follows:

$$\begin{aligned} S &= \mathrm{Obj}, \\ P(s) &= \Sigma_{s'}\, \mathcal{C}(s, s'), \\ o(s) &= (s, \mathrm{id}_s), \\ s \downarrow p &= p.1 \quad \text{(codomain projection)}, \\ p \oplus q &= (q.1,\ \mathrm{comp}(p.2, q.2)). \end{aligned}$$

Laws D1 and D4 hold by rfl (the codomain of a composite is the codomain of the second factor, definitionally). D2/D3/D5 are the category laws.

Lean.

```
def toDirectedContainer (C : SmallCat) : DirectedContainer where
  Shape := C.Obj
  Pos s := (s' : C.Obj) × C.Hom s s'
  root s := ⟨s, C.id s⟩
  sub _ p := p.1
  shift _ p q := ⟨q.1, C.comp p.2 q.2⟩
  d1 _ := rfl
  d4 _ _ _ := rfl
  d2 _ q := congrArg (Sigma.mk q.1) (C.id_comp q.2)
  d3 _ p := congrArg (Sigma.mk p.1) (C.comp_id p.2)
  d5 _ p q r := congrArg (Sigma.mk r.1) (C.assoc p.2 q.2 r.2)
```

Haskell.

```
-- Directed container from a small category.
data DCont obj = DCont
  { shape  :: obj
  , pos    :: obj → [(obj, Morphism)]   -- outgoing morphisms
  , root   :: obj → (obj, Identity)
```

```
  , sub    :: obj → (obj, Morphism) → obj    -- = fst (codomain)
  , shift  :: obj → (obj, Morphism) → (obj, Morphism) → (obj,
     Morphism)
  }
-- shift _ (s', f) (s'', g) = (s'', comp f g)
```

4 The round-trip, honestly

The two constructions are *not* definitionally mutually inverse. Going $D \mapsto \mathcal{C}(D) \mapsto D'$, the positions of D' at s are $\Sigma_{s'}\,\{p : P(s) \mid s \downarrow p = s'\}$ — each position tagged with its codomain — whereas D has plain $P(s)$. The two are in bijection (a position determines its codomain), exhibited below.

4.1 Definitions: `toRoundtripPos` and `ofRoundtripPos`

Definition 4.1 (Round-trip position bijection (forward))**.** Tag a position with its codomain:

$$\text{toRoundtripPos}(s, p) \;=\; (s \downarrow p,\; p,\; \text{rfl}).$$

Definition 4.2 (Round-trip position bijection (backward))**.** Forget the codomain tag:

$$\text{ofRoundtripPos}(s, (s', p, h)) \;=\; p.$$

Lean.

```
def toRoundtripPos (D : DirectedContainer) (s : D.Shape) (p : D.Pos s)
    :
    D.toSmallCat.toDirectedContainer.Pos s :=
  ⟨D.sub s p, p, rfl⟩

def ofRoundtripPos (D : DirectedContainer) (s : D.Shape)
    (p : D.toSmallCat.toDirectedContainer.Pos s) : D.Pos s :=
  p.2.val
```

Haskell.

```
toRoundtripPos :: Pos s → (Sub s, Pos s, ())    -- tag with codomain
toRoundtripPos p = (sub p, p, ())

ofRoundtripPos :: (Sub s, Pos s, ()) → Pos s    -- forget tag
ofRoundtripPos (_, p, _) = p
```

4.2 Theorems: round-trip bijection

Theorem 4.3 (Forward-backward is identity)**.**

$$\text{ofRoundtripPos}(s,\; \text{toRoundtripPos}(s, p)) \;=\; p.$$

Theorem 4.4 (Backward-forward is identity)**.**

$$\text{toRoundtripPos}(s,\; \text{ofRoundtripPos}(s, p)) \;=\; p.$$

Lean.

```
theorem ofRoundtripPos_toRoundtripPos (D : DirectedContainer)
    (s : D.Shape) (p : D.Pos s) :
    D.ofRoundtripPos s (D.toRoundtripPos s p) = p

theorem toRoundtripPos_ofRoundtripPos (D : DirectedContainer)
    (s : D.Shape) (p : D.toSmallCat.toDirectedContainer.Pos s) :
    D.toRoundtripPos s (D.ofRoundtripPos s p) = p
```

Haskell.

```haskell
-- prop_fwd_bwd p = ofRoundtripPos (toRoundtripPos p) == p
-- prop_bwd_fwd p = toRoundtripPos (ofRoundtripPos p) == p
```

5 Morphisms: directed-container morphisms and cofunctors

5.1 Structure: `DContMorphism`

Definition 5.1 (Directed-container morphism)**.** A **morphism of directed containers** $\varphi : C \Rightarrow D$ consists of:

- a *covariant* shape map $f : S_C \to S_D$;
- a *contravariant* position map $f_s^\sharp : P_D(f(s)) \to P_C(s)$;

satisfying:

- **M0:** $f(C{\downarrow_s} f_s^\sharp(h)) = D{\downarrow_{f(s)}} h$;
- **M1:** $f_s^\sharp(o_D(f(s))) = o_C(s)$;
- **M2:** $f_s^\sharp(h \oplus_D k) = f_s^\sharp(h) \oplus_C f^\sharp_{C{\downarrow_s} f_s^\sharp(h)}(M0^{-1} \triangleright k)$.

Lean.

```
structure DContMorphism (C D : DirectedContainer) where
  onShapes : C.Shape → D.Shape
  onPos : (s : C.Shape) → D.Pos (onShapes s) → C.Pos s
  m0 : ∀ (s : C.Shape) (h : D.Pos (onShapes s)),
    onShapes (C.sub s (onPos s h)) = D.sub (onShapes s) h
  m1 : ∀ s : C.Shape, onPos s (D.root (onShapes s)) = C.root s
  m2 : ∀ (s : C.Shape) (h : D.Pos (onShapes s))
    (k : D.Pos (D.sub (onShapes s) h)),
    onPos s (D.shift (onShapes s) h k)
      = C.shift s (onPos s h)
          (onPos (C.sub s (onPos s h)) ((m0 s h).symm [> k))
```

Haskell.

```haskell
data DContMorph c d = DContMorph
  { onShapes :: Shape c → Shape d       -- covariant
  , onPos    :: Shape c → Pos d → Pos c   -- contravariant
  -- M0: onShapes (sub_C s (onPos s h)) = sub_D (onShapes s) h
  -- M1: onPos s (root_D (onShapes s)) = root_C s
  -- M2: onPos s (shift_D (onShapes s) h k)
  --       = shift_C s (onPos s h)
  --           (onPos (sub_C s (onPos s h)) k)
  }
```

Fidelity note. M2 in Lean involves a transport: $k : P_D(D{\downarrow} f(s){\downarrow} h)$ must be transported along M0 to live over $f(C{\downarrow} f_s^\sharp(h))$ before it can be fed to $\oplus_D$. This transport is invisible in the Haskell rendering since Haskell does not track which shape a position belongs to at the type level.

5.2 Structure: `Cofunctor`

Definition 5.2 (Cofunctor)**.** A **cofunctor** $\varphi : \mathcal{C}(C) \to \mathcal{C}(D)$ consists of:

- an *object map* $\mathrm{obj} : S_C \to S_D$;
- a *lift* $\mathrm{lift}_s : P_D(\mathrm{obj}(s)) \to P_C(s)$ (morphisms lift *backwards*);

satisfying the laws C0/C1/C2, which are word-for-word M0/M1/M2 of a directed-container morphism. The two structures carry *literally the same data*.

Lean.

```
structure Cofunctor (C D : DirectedContainer) where
  obj : C.Shape → D.Shape
  lift : (s : C.Shape) → D.Pos (obj s) → C.Pos s
  c0 : ∀ (s : C.Shape) (h : D.Pos (obj s)),
    obj (C.sub s (lift s h)) = D.sub (obj s) h
  c1 : ∀ s : C.Shape, lift s (D.root (obj s)) = C.root s
  c2 : ∀ (s : C.Shape) (h : D.Pos (obj s))
    (k : D.Pos (D.sub (obj s) h)),
    lift s (D.shift (obj s) h k)
      = C.shift s (lift s h)
          (lift (C.sub s (lift s h)) ((c0 s h).symm [> k))
```

Haskell.

```
-- Literally the same data as DContMorph, different field names.
data Cofunctor c d = Cofunctor
  { obj  :: Shape c → Shape d
  , lift :: Shape c → Pos d → Pos c
  }
```

5.3 Definitions and theorems: the hom-set bijection

Definition 5.3 (From morphism to cofunctor)**.** `DContMorphism.toCofunctor`: the identity on data (M0/M1/M2 become C0/C1/C2 by renaming).

Definition 5.4 (From cofunctor to morphism)**.** `Cofunctor.toDContMorphism`: the identity on data (C0/C1/C2 become M0/M1/M2 by renaming).

Theorem 5.5 (Bijection, one way)**.** φ`.toCofunctor.toDContMorphism` $= \varphi$ (by `rfl`).

Theorem 5.6 (Bijection, other way)**.** φ`.toDContMorphism.toCofunctor` $= \varphi$ (by `rfl`).

Lean.

```
def DContMorphism.toCofunctor {C D : DirectedContainer}
    (phi : DContMorphism C D) : Cofunctor C D where
  obj := phi.onShapes;  lift := phi.onPos
  c0 := phi.m0;  c1 := phi.m1;  c2 := phi.m2

def Cofunctor.toDContMorphism {C D : DirectedContainer}
    (phi : Cofunctor C D) : DContMorphism C D where
  onShapes := phi.obj;  onPos := phi.lift
  m0 := phi.c0;  m1 := phi.c1;  m2 := phi.c2

theorem DContMorphism.toDContMorphism_toCofunctor {C D :
   DirectedContainer}
    (phi : DContMorphism C D) : phi.toCofunctor.toDContMorphism = phi

theorem Cofunctor.toCofunctor_toDContMorphism {C D : DirectedContainer
   }
    (phi : Cofunctor C D) : phi.toDContMorphism.toCofunctor = phi
```

Haskell.

```
-- Both conversions are identity functions on the underlying data.
toCofunctor       :: DContMorph c d → Cofunctor c d
toCofunctor m = Cofunctor (onShapes m) (onPos m)

toDContMorphism   :: Cofunctor c d → DContMorph c d
```

```
toDContMorphism f = DContMorph (obj f) (lift f)
```

6 The category of directed containers (via cofunctors)

6.1 Definition: `Cofunctor.id`

Definition 6.1 (Identity cofunctor)**.** The identity cofunctor on C: object map is id, lift is id. All laws hold by rfl.

Lean.

```
def Cofunctor.id (C : DirectedContainer) : Cofunctor C C where
  obj s := s
  lift _ p := p
  c0 _ _ := rfl;  c1 _ := rfl;  c2 _ _ _ := rfl
```

Haskell.

```
idCofunctor :: Cofunctor c c
idCofunctor = Cofunctor id (\_ p → p)
```

6.2 Lemma: `Cofunctor.comp_transport`

Proposition 6.2 (Transport commutes with composition)**.** Let $\psi : \mathcal{C}(D) \to \mathcal{C}(E)$. Given shape equalities $e : y = x$ in D and $e_2 : \psi.\mathrm{obj}(x) = z$ in E, and a position $k : P_E(z)$:

$$e^{-1} \triangleright \psi.\mathrm{lift}_x(e_2^{-1} \triangleright k) \;=\; \psi.\mathrm{lift}_y((\mathrm{congrArg}(\psi.\mathrm{obj}, e) \cdot e_2)^{-1} \triangleright k).$$

This is the key dependent step enabling composition of cofunctors.

Lean.

```
theorem Cofunctor.comp_transport {D E : DirectedContainer} (psi :
    Cofunctor D E)
    {x y : D.Shape} {z : E.Shape} (e : y = x) (e2 : psi.obj x = z)
    (k : E.Pos z) :
    e.symm [> psi.lift x (e2.symm [> k)
      = psi.lift y (((congrArg psi.obj e).trans e2).symm [> k)
```

Haskell.

```
-- Transport-coherence lemma; collapses to rfl in Lean.
-- No direct Haskell analogue (transports are trivial without
    dependent types).
```

6.3 Definition: `Cofunctor.comp`

Definition 6.3 (Composition of cofunctors)**.** Given cofunctors $\varphi : C \to D$ and $\psi : D \to E$, their composite $\psi \circ \varphi : C \to E$ has:

$$\mathrm{obj}(s) = \psi.\mathrm{obj}(\varphi.\mathrm{obj}(s)),$$
$$\mathrm{lift}_s(m) = \varphi.\mathrm{lift}_s(\psi.\mathrm{lift}_{\varphi.\mathrm{obj}(s)}(m)).$$

The lift composes *backwards* (positions pulled through ψ then φ).

Lean.

```
def Cofunctor.comp {C D E : DirectedContainer}
    (phi : Cofunctor C D) (psi : Cofunctor D E) : Cofunctor C E where
  obj s := psi.obj (phi.obj s)
  lift s m := phi.lift s (psi.lift (phi.obj s) m)
```

```
  c0 s m := (congrArg psi.obj (phi.c0 s (psi.lift (phi.obj s) m))).
     trans
    (psi.c0 (phi.obj s) m)
  c1 s := by rw [psi.c1, phi.c1]
  c2 s h k := by
    rw [psi.c2, phi.c2,
      psi.comp_transport (phi.c0 s (psi.lift (phi.obj s) h)) (psi.c0 (
         phi.obj s) h)]
```

Haskell.

```
compCofunctor :: Cofunctor c d → Cofunctor d e → Cofunctor c e
compCofunctor phi psi = Cofunctor
  { obj  = obj psi . obj phi
  , lift = \s m → lift phi s (lift psi (obj phi s) m)
  }
```

6.4 Theorems: category laws for `Cofunctor`

Theorem 6.4 (Left unit)**.** $\mathrm{comp}(\mathrm{id}_C, \varphi) = \varphi$ (by `rfl`).

Theorem 6.5 (Right unit)**.** $\mathrm{comp}(\varphi, \mathrm{id}_D) = \varphi$ (by `rfl`).

Theorem 6.6 (Associativity)**.** $\mathrm{comp}(\mathrm{comp}(\varphi, \psi), \chi) = \mathrm{comp}(\varphi, \mathrm{comp}(\psi, \chi))$ (by `rfl`).

Lean.

```
theorem Cofunctor.id_comp {C D : DirectedContainer} (phi : Cofunctor C
    D) :
    comp (id C) phi = phi

theorem Cofunctor.comp_id {C D : DirectedContainer} (phi : Cofunctor C
    D) :
    comp phi (id D) = phi

theorem Cofunctor.comp_assoc {B C D E : DirectedContainer}
    (phi : Cofunctor B C) (psi : Cofunctor C D) (chi : Cofunctor D E)
       :
    comp (comp phi psi) chi = comp phi (comp psi chi)
```

Haskell.

```
-- prop_id_comp phi = compCofunctor idCofunctor phi == phi
-- prop_comp_id phi = compCofunctor phi idCofunctor == phi
-- prop_assoc phi psi chi =
--   compCofunctor (compCofunctor phi psi) chi
--     == compCofunctor phi (compCofunctor psi chi)
```

7 Directed-container morphisms as a category

7.1 Definitions and theorems: `DContMorphism` as a category

Definition 7.1 (Identity directed-container morphism)**.** id_C = (Cofunctor.id C).toDContMorphism — the identity on data.

Definition 7.2 (Composition of directed-container morphisms)**.** $\mathrm{comp}(\varphi, \psi)$ = (φ.toCofunctor$\circ$ ψ.toCofunctor).toDContMorphism.

Theorem 7.3 (`toCofunctor` preserves identities)**.** (id C).toCofunctor = Cofunctor.id C (by `rfl`).

Theorem 7.4 (`toCofunctor` preserves composition)**.** $(\varphi \cdot \psi)$.toCofunctor $=$ φ.toCofunctor $\circ$ ψ.toCofunctor (by `rfl`).

Together with the `rfl` bijection, this makes `toCofunctor` an isomorphism of categories between directed-container morphisms and cofunctors.

Lean.

```
def DContMorphism.id (C : DirectedContainer) : DContMorphism C C :=
  (Cofunctor.id C).toDContMorphism

def DContMorphism.comp {C D E : DirectedContainer}
    (phi : DContMorphism C D) (psi : DContMorphism D E) :
       DContMorphism C E :=
  (phi.toCofunctor.comp psi.toCofunctor).toDContMorphism

theorem DContMorphism.toCofunctor_id (C : DirectedContainer) :
    (id C).toCofunctor = Cofunctor.id C

theorem DContMorphism.toCofunctor_comp {C D E : DirectedContainer}
    (phi : DContMorphism C D) (psi : DContMorphism D E) :
    (phi.comp psi).toCofunctor = phi.toCofunctor.comp psi.toCofunctor
```

Haskell.

```
idDCM :: DContMorph c c
idDCM = toDContMorphism idCofunctor

compDCM :: DContMorph c d → DContMorph d e → DContMorph c e
compDCM phi psi = toDContMorphism (compCofunctor (toCofunctor phi) (
   toCofunctor psi))
```

8 Translation conventions

1. **Dependent position maps.** M2 and C2 involve transporting a position along a shape equality (M0/C0) before it can be consumed by shift. Haskell renders this as a plain function application; the type-level correctness guarantee is lost.
2. **Identity bijection.** The isomorphism between `DContMorphism` and `Cofunctor` is witnessed by `rfl` in both directions. This means the two Lean structures carry bit-for-bit identical data; the distinction is purely conceptual ("container morphism" vs. "cofunctor").
3. **Round-trip gap.** The equivalence DCont $\simeq$ Cat (Ahman–Uustalu 2016, Thm 3.6) holds only up to bijection on position types, not definitional equality. The file exhibits the bijection explicitly rather than claiming a `rfl`-equality that does not hold.
4. **Proof bodies omitted.** Only statements are translated.